

Journée 2

Architecture Micro-services

Section 3 · TP Symfony Avis-Clients (S7)

Nom / Prénom : DANILO Elouan

3. TP Symfony Avis-Clients, différences avec Django

Le service **Avis-Clients** est codé en **Symfony 7** avec **API Platform**. Il expose une entité `Review` avec un CRUD complet auto-généré, de la validation, un appel `HttpClient` au Catalogue, des tests `PHPUnit` et une documentation `Swagger` gratuite. La base est **MariaDB** (contre `PostgreSQL` pour le Catalogue), ce qui démontre le polyglottisme de persistance.

La « magie » d'API Platform

Une seule annotation `#[ApiResponse]` sur l'entité génère tout le CRUD REST et la doc OpenAPI. La validation se déclare par des attributs posés directement sur les propriétés.

```
#[ORM\Entity(repositoryClass: ReviewRepository::class)]
#[ApiResponse(operations: [
    new GetCollection(), new Get(),
    new Post(processor: ReviewPersistProcessor::class),
    new Patch(processor: ReviewPersistProcessor::class),
    new Delete(),
])]
class Review
{
    #[Assert\Range(min: 1, max: 5, notInRangeMessage: 'Note entre {{ min }} et {{ max }}')]
    private ?int $rating = null;

    #[Assert\NotBlank]
    #[Assert\Length(min: 5, max: 1000)]
    private ?string $comment = null;

    #[Assert\Positive]
    private ?int $productId = null;
    // ... author, createdAt
}
```

API Platform expose alors automatiquement cinq opérations REST, à savoir `GET /api/reviews` (liste paginée), `POST /api/reviews` (création), `GET /api/reviews/{id}` (détail), `PATCH /api/reviews/{id}` (modification) et `DELETE /api/reviews/{id}` (suppression), le tout documenté en OpenAPI.

Appel inter-services avec HttpClient et State Processor

La vérification du produit est branchée dans un **State Processor** qui décore le `persist` de Doctrine. Avant d'écrire, on interroge le Catalogue via le **HttpClient** de Symfony.

```
public function process(mixed $data, Operation $op, array $uriVars = [], array $ctx = []): mixed
{
    if ($data instanceof Review && !$this->catalogue->productExists($data->getProductId())) {
        throw new HttpException(422, 'Produit inexistant : aucun produit ... dans le Catalogue.');
```

```

    }
    return $this->persistProcessor->process($data, $op, $uriVars, $ctx);
}

```

Tableau comparatif Django/DRF ↔ Symfony/API Platform

Thème	Django / DRF	Symfony / API Platform
Génération API	ViewSet + Serializer + routeur à écrire	# [ApiResource] → CRUD auto
Documentation	drf-spectacular à configurer	Swagger gratuite sur /api/docs
ORM	Django ORM	Doctrine (attributs # [ORM])
Validation	validate_* du serializer	Attributs # [Assert\...]
Réponses	JSON simple	JSON-LD / Hydra (HATEOAS)
Erreurs	détail DRF (400)	RFC 7807 (/api/errors/422)
Appels sortants	requests	Symfony HttpClient
Dépendances	pip / requirements.txt	Composer / composer.json
Brique de code	« app » Django	« bundle » Symfony
Hook d'écriture	perform_create()	State Processor

Tests

8 tests PHPUnit passent (WebTestCase pour l'API et KernelTestCase pour la validation). Ils couvrent la liste, la création 201, le produit inexistant 422, la note hors bornes 422 et le commentaire vide 422. Le Catalogue est remplacé par un double de test pour ne pas dépendre du Django réel.

Difficultés rencontrées et solutions

Le code du service étant fourni, l'essentiel du travail a porté sur la mise en route de l'environnement et la résolution des problèmes de configuration.

Difficulté	Solution mise en place
PHP, Composer et le Symfony CLI absents de la machine.	Installation de PHP 8.3 portable + Composer en espace utilisateur (sans droits administrateur), ajout au PATH.
Composer refuse d'installer Symfony 7.1 (avis de sécurité).	Montée de version en Symfony 7.4 / API Platform 4.2 ; adaptation de la clé de config docformats → docs_formats.
Composer ne décompresse pas les paquets (The zip extension is missing).	Activation de extension=zip dans php.ini.
La Swagger UI ne s'affiche pas (page cassée, logo absent, CSS/JS en 404).	(1) installation de twig-bundle (template de la doc manquant); (2) format html remis dans formats; (3) routeur public/router.php pour servir les assets (sinon php -S les renvoie en text/html).
config/routes.yaml pointe vers un dossier src/Controller inexistant.	Création du dossier (service API-only, sans contrôleur).
Aucun serveur MariaDB installé en local.	Exécution sur SQLite via .env.local (et .env.test.local pour les tests); la configuration MariaDB du TP est conservée pour Docker.
assertJsonContains() indisponible en WebTestCase.	Décodage JSON manuel (json_decode) puis assertions sur les champs.

Livrables

Le code, la collection Postman (`AvisClients.postman_collection.json`) et le présent rapport sont versionnés dans le dépôt Git <https://github.com/Oasio/micro-service>. La documentation interactive Swagger est servie sur `/api/docs` (capture ci-dessous).

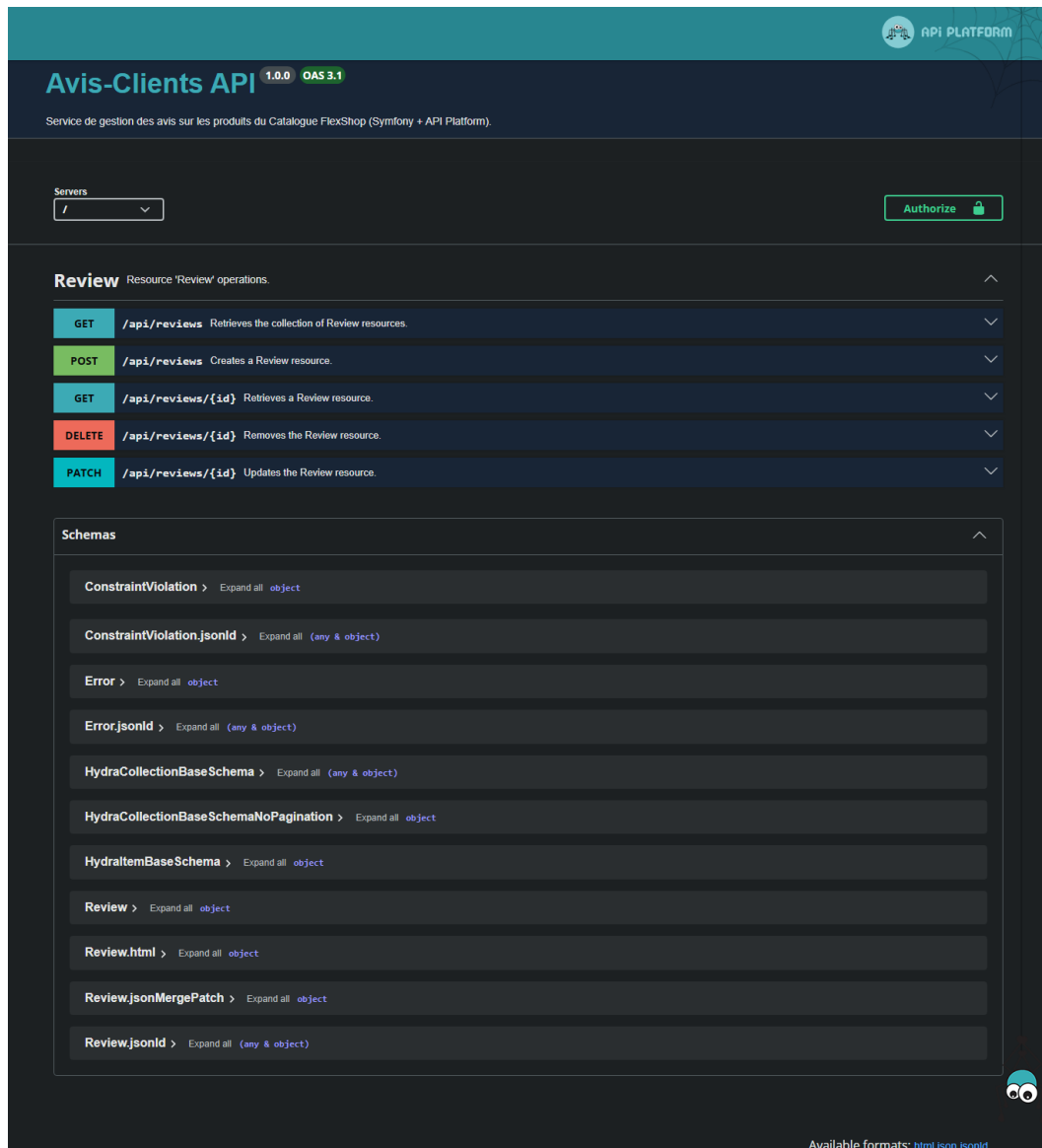


FIGURE 1 – Swagger UI d’Avis-Clients sur `http://localhost:8001/api/docs`. Le CRUD complet est auto-généré par API Platform, avec des schémas JSON-LD/Hydra.